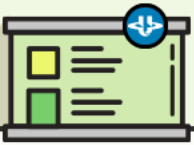


2026년 1학기

AI를 위한 수학의 응용 출석수업

통계·데이터과학과 박준희



출석수업안내

- 1 강의자료 : 박준희 교수 홈페이지자료실
- 2 질문 : 박준희 교수 홈페이지상담게시판
- 3 강의범위 : 교재 1 ~ 4장, 10~11장

0

오늘의 핵심 주제

● 다변량 정규분포와 주성분분석(PCA)

■ 다변량 정규분포와 AI

- 현실의 데이터는 대부분 여러 변수의 결합 구조
- 많은 AI 모델이 정규분포 가정을 기반으로 작동
- 노이즈 모델링의 기본 전제
- 간결한 확률적 구조: 평균벡터와 공분산행렬

■ 그러나 실제 데이터 분석에서는 ...

- 변수가 많아 구조 파악이 어려움
- 서로 비슷한 변수들이 많음
- 데이터를 시각화하기엔 차원이 너무 높음
- 정보를 유지하면서 단순화하고 싶음

} 핵심 구조 추출(차원 축소)

0

오늘의 핵심 주제

● 다변량 정규분포와 주성분분석(PCA)

■ 진행순서

- 벡터와 행렬 데이터를 표현하는 언어
- 고윳값 분해 차원 축소를 위한 핵심 도구
- 다변량 정규분포 데이터의 구조 파악
- 주성분분석 차원 축소 이론과 실습



벡터와 행렬



1 벡터와 행렬

● 벡터란 무엇인가?

- n 개의 수치 성분으로 이루어진 순서있는 나열
 - 예) 각 학생의 중간, 기말 시험점수

학생	중간	기말	벡터 표현
<i>A</i>	70	85	(70, 85)
<i>B</i>	90	60	(90, 60)
<i>C</i>	50	50	(50, 50)

- A 학생의 중간, 기말 점수가 (70, 85)라면, 이것이 하나의 벡터
- 숫자가 2개면 2차원 벡터, 10개면 10차원 벡터
- 데이터 한 건 = 벡터 하나

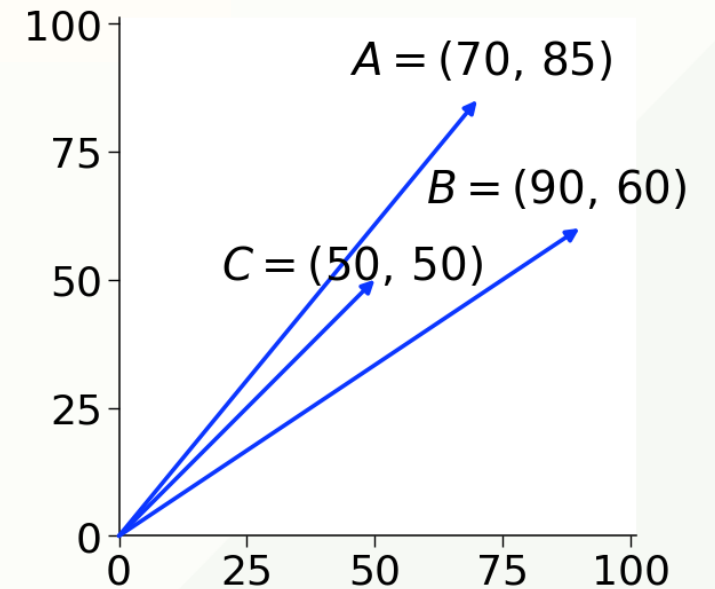
1 벡터와 행렬

● 벡터의 기하학적 해석

- 벡터: 좌표 위의 점(또는 화살표)으로 해석 가능
 - 예) 각 학생의 중간, 기말 시험점수

학생	중간	기말	벡터 표현
<i>A</i>	70	85	(70, 85)
<i>B</i>	90	60	(90, 60)
<i>C</i>	50	50	(50, 50)

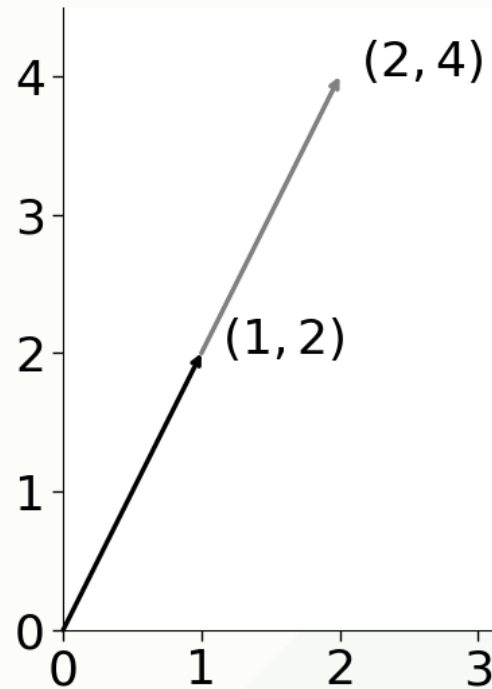
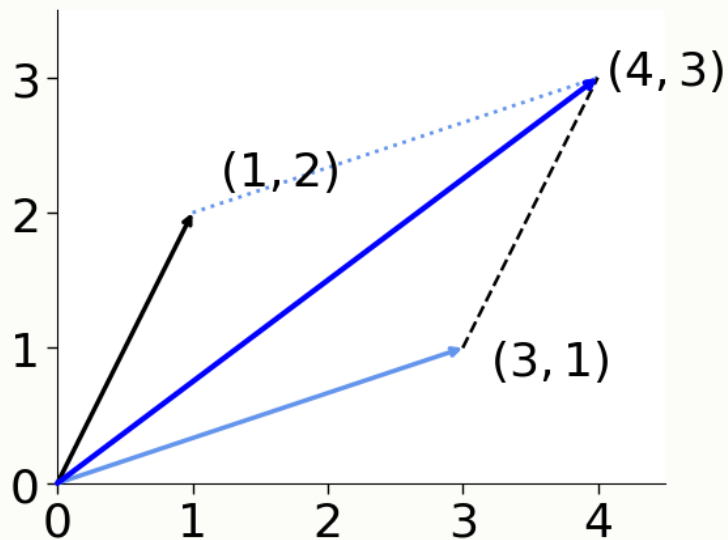
- 가로축: 중간시험 점수
- 세로축: 기말시험 점수



1 벡터와 행렬

● 벡터의 연산과 스칼라 곱

- 벡터의 덧셈: 같은 위치의 숫자끼리 더함
 - $(1, 2) + (3, 1) = (4, 3)$
- 스칼라 곱: 모든 숫자에 같은 수를 곱함
 - $2 \times (1, 2) = (2, 4)$



1 벡터와 행렬

● 벡터의 크기

- 벡터의 길이: 원점과점사이의거리

- 예) 벡터 $(3, 4)$ 의 L_2 노름

$$\|(3, 4)\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$$

- 예) 위 벡터의 L_1 노름

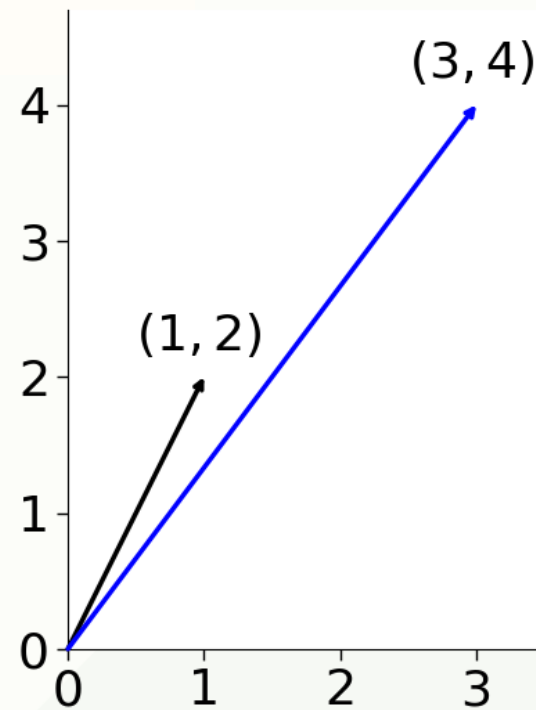
$$\|(3, 4)\|_1 = |3| + |4| = 7$$

- 두 벡터사이의거리: 두점사이의거리

- $d(v, u) = \|v - u\|$

- 예) 벡터 $(1, 2), (3, 4)$ 사이의 거리

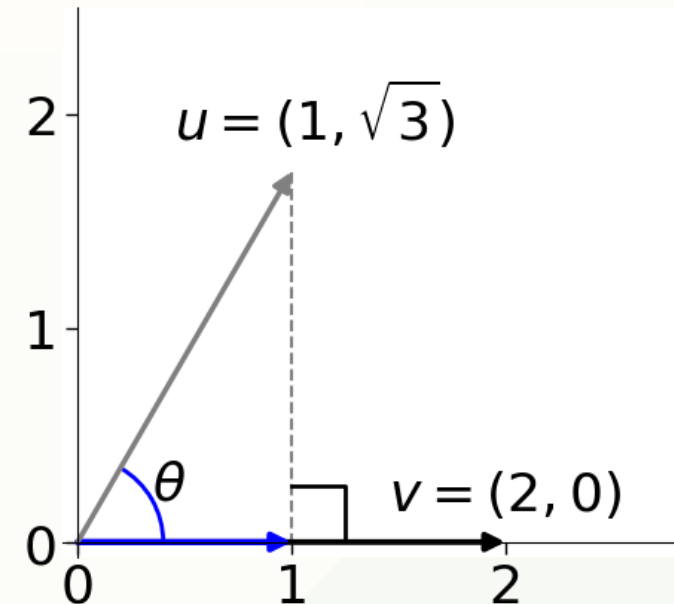
$$\begin{aligned} d((1, 2), (3, 4)) &= \|(1, 2) - (3, 4)\| = \|(-2, -2)\| \\ &= \sqrt{(-2)^2 + (-2)^2} = \sqrt{8} \\ &= 2\sqrt{2} \end{aligned}$$



1 벡터와 행렬

● 벡터의 내적

- 계산: 같은 위치 원소끼리 곱해서 모두 더함
 - $(1, \sqrt{3}) \cdot (2, 0) = 1 \times 2 + \sqrt{3} \times 0 = 2$
두 벡터의 크기 & 사잇각 θ 의 코사인 값을 곱함
 - $(1, \sqrt{3}) \cdot (2, 0) = 2 \times 2 \times \cos 60^\circ = 2$
- 해석: 두 벡터의 크기 & 방향 유사성
 - 두 벡터가 같은 방향: 양수
 - 수직(직교): 0
 - 반대 방향: 음수



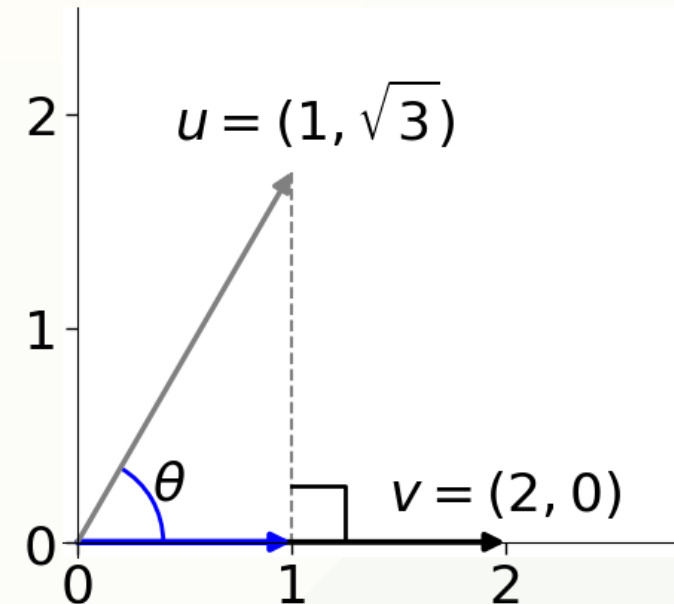
1 벡터와 행렬

● 코사인 유사도

- 계산: 두 벡터의 사이각 θ 의 코사인값

$$\cos \theta = \frac{v \cdot u}{\|v\| \|u\|}$$

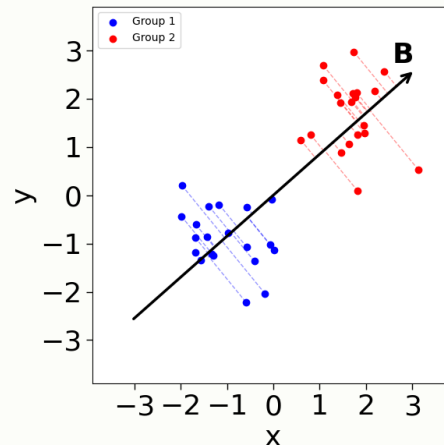
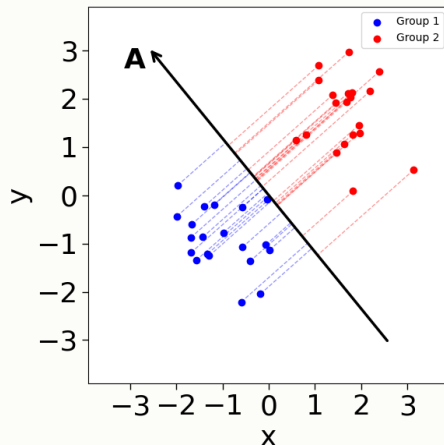
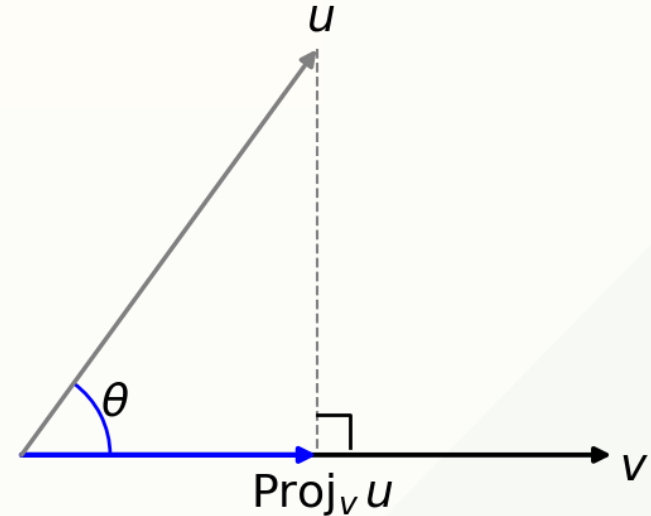
- 해석: 두 벡터의 방향 유사성
 - 두 벡터가 같은 방향: 양수
 - 수직(직교): 0
 - 반대 방향: 음수



1 벡터와 행렬

● 벡터의 정사영

- 한 벡터 위에 다른 벡터의 그림자를 내리는 것
 - 예) 벡터 u 를 벡터 v 로 사영(투영)
벡터 u 에서 v 방향으로 수선을 내림
- “어떤 축 위에 데이터를 투영하면 어떤 모습이 될까?”
 - 축 A에 투영: 점들이 뭉쳐서 구분 안 됨
 - 축 B에 투영: 점들이 넓게 퍼져서 구분 잘 됨



1 벡터와 행렬

● 행렬이란 무엇인가?

- 숫자들을 직사각형 형태로 배열한 것

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} = [a_{ij}] \in \mathbb{R}^{m \times n}$$

- 예) 학생들의 중간, 기말 시험점수(3행 2열 행렬)

학생	중간	기말	행렬 표현
A	70	85	$\begin{bmatrix} 70 & 85 \\ 90 & 60 \\ 50 & 50 \end{bmatrix}$
B	90	60	
C	50	50	

1 벡터와 행렬

● 행렬의 기본 연산

■ 행렬의 덧셈과 뺄셈

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \pm \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} a \pm e & b \pm f \\ c \pm g & d \pm h \end{bmatrix}$$

■ 행렬의 스칼라 곱

$$s \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} sa & sb \\ sc & sd \end{bmatrix}$$

■ 행렬의 전치

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^T = \begin{bmatrix} a & c \\ b & d \end{bmatrix}$$

$$\text{■ 대칭행렬: } A^T = A$$

1 벡터와 행렬

● 행렬 곱

- 벡터를 다른 벡터로 변환시키는 함수로 해석 가능
- 왼쪽 행렬의 열과 오른쪽 행렬의 행 수가 같아야 연산 성립
- 비가환 연산: 일반적인 곱셈과 달리 교환 법칙이 성립하지 않음

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

$$\begin{bmatrix} e & f \\ g & h \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} ae + cf & be + df \\ ag + ch & bg + dh \end{bmatrix}$$

- 행렬 곱의 성질

- 결합 법칙: $(AB)C = A(BC)$
- 분배 법칙: $A(B + C) = AB + AC$
- 항등 행렬 I_n (또는 I_m)에 대해

$$AI_n = A, I_m A = A$$

1 벡터와 행렬

● 행렬식

- 정리: 정사각행렬에 대응하는 하나의 스칼라값으로, 다음과 같이 전개 (i 번째 행 기준)

$$\det(A) = \sum_{j=1}^n (-1)^{(i+j)} a_{ij} \det(M_{ij})$$

- M_{ij} : 행렬 A 에서 i 번째 행과 j 번째 열을 제거한 부분행렬
 - 임의의 j 번째 열을 기준으로도 비슷하게 전개 가능
- 해석: 해당 변환이 가져오는 부피(or면적) 변화량
- 2×2 행렬의 행렬식

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \Rightarrow \det(A) = ad - bc$$

1 벡터와 행렬

● 역행렬

- 정의를: 정사각행렬 $A \in \mathbb{R}^{n \times n}$ 에 대해 다음을 만족하는 행렬 A^{-1}

$$AA^{-1} = A^{-1}A = I_n$$

- I_n : $n \times n$ 항등행렬
- 해석: 행렬 곱에 대한 역원
 - 행렬 A 가 가한 변환을 되돌리는 행렬
- 2×2 행렬의 역행렬

$$A^{-1} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

1 벡터와 행렬

● 선형변환

- 벡터 공간의 관점: 한 벡터를 다른 벡터로 변환하는 함수
- 정의: 다음 두 조건을 만족하는 함수 T

- 덧셈에 대한 보존

$$T(\mathbf{v} + \mathbf{u}) = T(\mathbf{v}) + T(\mathbf{u}) \quad \forall \mathbf{v}, \mathbf{u} \in \mathbb{R}^n$$

- 스칼라 곱에 대한 보존

$$T(s\mathbf{v}) = sT(\mathbf{v}) \quad \forall s \in \mathbb{R}, \mathbf{v} \in \mathbb{R}^n$$

- 이러한 선형변환은 행렬 $A \in \mathbb{R}^{m \times n}$ 로 표현가능

$$T(\mathbf{v}) = A\mathbf{v}$$

1

벡터와 행렬

● 선형변환의 연속 적용

- 변환 A 를 한번, 변환 B 를 한번 $\rightarrow$ 합쳐서 BA

$$B(Av) = (BA)v$$

- 먼저 A 로 변환하고, 그 다음 B 로 변환하는 것
- 한 번에 BA 라는 행렬로 변환하는 것
- 두 결과가 같음: 선형변환은 아무리 많이 적용해도 결국 하나의 행렬
 $\rightarrow$ 신경망에서 활성화 함수가 필요한 근거

1 벡터와 행렬

● 행렬의 랭크(개략적 설명)

- 랭크의 직관적 해석: 행렬이 가지는 독립적인 성분의 개수

$$\text{rank}(A) = \dim(\text{Col}(A)) = \dim(\text{Row}(A))$$

- $\text{Col}(A)$: 행렬 A 의 열공간
- $\text{Row}(A)$: 행렬 A 의 행공간
- 랭크를 구하는 직관적 방법: 가우스 소거법 → 가우스 행렬(REF) 확보
 - 리딩항 1의 개수 = 행렬의 랭크
- 가우스 소거법을 위한 기본행연산
 - 행 교환: $R_i \leftrightarrow R_j \ (i \neq j)$
 - 행 스칼라배: $R_i \leftarrow cR_i \ (c \neq 0)$
 - 다른 행의 스칼라배 더하기: $R_i \leftarrow R_i + cR_j \ (i \neq j, c \neq 0)$

1 벡터와 행렬

● 랭크, 역행렬, 행렬식의 관계

- 가우스-조던 소거법: 기약가우스 행렬(RREF) 확보

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 7 \\ 1 & 1 & 1 \end{bmatrix}$$

- 확장행렬: $[A | I] = \left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 2 & 4 & 7 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 \end{array} \right]$

- $R_2 \leftarrow R_2 - 2R_1, R_3 \leftarrow R_3 - R_1$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 0 & 1 & -2 & 1 & 0 \\ 0 & -1 & -2 & -1 & 0 & 1 \end{array} \right]$$

1 벡터와 행렬

● 랭크, 역행렬, 행렬식의 관계

- 가우스-조던 소거법: 기약가우스행렬(RREF) 확보

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 0 & 1 & -2 & 1 & 0 \\ 0 & -1 & -2 & -1 & 0 & 1 \end{array} \right]$$

- $R_3 \leftarrow -R_3$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 0 & 1 & -2 & 1 & 0 \\ 0 & 1 & 2 & 1 & 0 & -1 \end{array} \right]$$

- $R_2 \leftrightarrow R_3$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 & -1 \\ 0 & 0 & 1 & -2 & 1 & 0 \end{array} \right]$$

1 벡터와 행렬

● 랭크, 역행렬, 행렬식의 관계

- 가우스-조던 소거법: 기약가우스 행렬(RREF) 확보

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 & -1 \\ 0 & 0 & 1 & -2 & 1 & 0 \end{array} \right]$$

- $R_1 \leftarrow R_1 - 2R_2$

$$\left[\begin{array}{ccc|ccc} 1 & 0 & -1 & -1 & 0 & 2 \\ 0 & 1 & 2 & 1 & 0 & -1 \\ 0 & 0 & 1 & -2 & 1 & 0 \end{array} \right]$$

- $R_1 \leftarrow R_1 + R_3, R_2 \leftarrow R_2 - 2R_3$

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & -3 & 1 & 2 \\ 0 & 1 & 0 & 5 & -2 & -1 \\ 0 & 0 & 1 & -2 & 1 & 0 \end{array} \right]$$

1 벡터와 행렬

● 랭크, 역행렬, 행렬식의 관계

- 가우스-조던 소거법: 역행렬이 없는 경우

$$B = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{bmatrix}$$

- 확장행렬: $[B | I] = \left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 2 & 4 & 6 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 \end{array} \right]$

- $R_2 \leftarrow R_2 - 2R_1, R_3 \leftarrow R_3 - R_1$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 0 & 0 & -2 & 1 & 0 \\ 0 & -1 & -2 & -1 & 0 & 1 \end{array} \right]$$

1 벡터와 행렬

● 랭크, 역행렬, 행렬식의 관계

- 가우스-조던 소거법: 역행렬이 없는 경우

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 0 & 0 & -2 & 1 & 0 \\ 0 & -1 & -2 & -1 & 0 & 1 \end{array} \right]$$

- $R_3 \leftarrow -R_3$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 0 & 0 & -2 & 1 & 0 \\ 0 & 1 & 2 & 1 & 0 & -1 \end{array} \right]$$

- $R_2 \leftrightarrow R_3$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 & -1 \\ 0 & 0 & 0 & -2 & 1 & 0 \end{array} \right]$$

02

고삐값 분해

2

고윳값 분해

● 고윳값과 고유벡터

- 행렬로 벡터를 변환하면: 일반적으로 방향도 바뀌고 크기도 바뀜
- 고유벡터: 변환을 해도 방향이 바뀌지 않는 특별한 벡터

- 예) $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$

$$A \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$A \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

- 고윳값: 해당 방향으로 늘어나는 정도
 - 고유벡터: $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ 방향
 - 고윳값: 3 (이 방향으로 3배 늘어남)
- 고윳값이 큰 방향: 데이터가 많이 퍼져 있는 (정보가 많은) 방향!

2 고윳값 분해

● 고윳값과 고유벡터

- 정사각행렬 $A \in \mathbb{R}^{n \times n}$ 와 영벡터가 아닌 v 가 있을 때 다음을 만족하면

$$Av = \lambda v$$

- v : A 의 고유벡터(eigenvector)
 - λ : v 의 고윳값(eigenvalue)
- 특성방정식(characteristic equation)

- 고윳값을 구하는 데 사용되는 공식

$$\det(A - \lambda I) = 0$$

- 고유벡터를 구하는 방법

- 특성방정식으로부터 구한 고윳값에 대해

$$(A - \lambda I)v = 0$$

를 만족하는 v 를 구함

2

고윳값 분해

● 고윳값 분해(EVD, Eigenvalue Decomposition)

- 정사각행렬 $A \in \mathbb{R}^{n \times n}$ 가 고윳값 분해가 가능하다면 다음과 같이 분해됨

$$A = PDP^{-1}$$

- $P = [v_1, v_2, \dots, v_n]$: 고유벡터들을 열벡터로 모은 행렬
 - P^{-1} 존재 = P 는 풀랭크행렬 = 고유벡터들은 선형독립
- $D = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$: 고윳값들을 대각에 둔 대각행렬
 - 고윳값들은 고유벡터들의 순서와 일치

2 고윳값 분해

● 고윳값 분해(EVD, Eigenvalue Decomposition)

- 파이썬(NumPy)을 이용한 고윳값 분해

```
# 예제 4-1
A = np.array([[4, 2],
              [1, 3]])
print(A)
```

```
[[4 2]
 [1 3]]
```

```
# 고윳값(lam)과 고유벡터(P) 계산
lam, P = np.linalg.eig(A)
print(lam)
print(P)
```

```
[5. 2.]
[[ 0.894427 -0.707107]
 [ 0.447214  0.707107]]
```

```
#  $PDP^{-1}$ 로 A 재건
P @ np.diag(lam) @ np.linalg.inv(P)
```

```
array([[4., 2.],
       [1., 3.]])
```

2 고윳값 분해

● 대칭행렬의 고윳값 분해

- 대칭인 정사각행렬 $A \in \mathbb{R}^{n \times n}$ 는 항상 다음과 같은 고윳값 분해가 가능

$$A = QDQ^T$$

- $D = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$: 고윳값 대각행렬
- $Q = [\hat{v}_1, \hat{v}_2, \dots, \hat{v}_n]$: 직교행렬(orthogonal matrix)
 - 서로 직교하고 길이가 1인 고유벡터들을 열벡터로 모은 행렬
 - $Q^{-1} = Q^T$ 성질을 가짐, $Q^T Q = Q Q^T = I$
- 공분산행렬은 항상 대칭행렬
 - 고윳값 분해 시 QDQ^T 사용

2 고윳값 분해

● 대칭행렬의 고윳값 분해

- 파이썬(NumPy)을 이용한 대칭행렬의 고윳값 분해

예제 4-6

```
A = np.array([[4, 0, 1],
              [0, 4, 0],
              [1, 0, 4]])
```

```
print(A)
```

```
[[4 0 1]
 [0 4 0]
 [1 0 4]]
```

QDQ^T 로 A 재건

```
Q @ np.diag(lam) @ Q.T
```

```
array([[4., 0., 1.],
       [0., 4., 0.],
       [1., 0., 4.]])
```

고윳값(lam)과 직교행렬(Q) 계산

```
lam, Q = np.linalg.eig(A)
```

```
print(lam)
```

```
print(Q)
```

```
[5. 3. 4.]
[[ 0.707107 -0.707107  0.   ]
 [ 0.         0.         1.   ]
 [ 0.707107  0.707107  0.   ]]
```


03

다변량 정규분포

3 다변량 정규분포

● 확률 벡터

- 2개 이상의 확률변수를 벡터 형태로 나타낸 것

$$\mathbf{X} = \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix}$$

- 평균 벡터: 확률 벡터 내 확률변수들의 기댓값을 모은 벡터

$$\boldsymbol{\mu} = \begin{bmatrix} E[X_1] \\ E[X_2] \\ E[X_3] \end{bmatrix}$$

- 공분산행렬: 확률 벡터 내 확률변수들의 분산과 공분산을 모은 행렬

$$\Sigma = \begin{bmatrix} \text{Var}[X_1] & \text{Cov}[X_1, X_2] & \text{Cov}[X_1, X_3] \\ \text{Cov}[X_2, X_1] & \text{Var}[X_2] & \text{Cov}[X_2, X_3] \\ \text{Cov}[X_3, X_1] & \text{Cov}[X_3, X_2] & \text{Var}[X_3] \end{bmatrix}$$

3 다변량 정규분포

● 확률 벡터

- 다변량 확률분포: 2개 이상의 확률변수에 대해 확률을 적절하게 분배하는 규칙
 - 연속형: 결합확률밀도함수(Joint PDF)
 - 이산형: 결합확률질량함수(Joint PMF)

● 다변량 정규분포

- 벡터 $\mathbf{X} \in \mathbb{R}^p$ 의 임의의 선형결합 $\mathbf{a}^T \mathbf{X}$ 가 정규분포를 따를 때 $\mathbf{X}$ 가 따르는 분포
 - 정규분포의 다변량 버전으로 다변량 데이터 모델링에 가장 널리 쓰임
 - 다변량 정규분포 $\mathbf{X} \sim N(\boldsymbol{\mu}, \Sigma)$ 의 결합확률밀도함수

$$f(\mathbf{x}) = (2\pi)^{-p/2} \det(\Sigma)^{-1/2} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})}$$

- 주의) $\mathbf{X}$ 의 각 성분이 정규분포라고 무조건 다변량 정규분포는 아님

3 다변량 정규분포

● 다변량 정규분포의 특징

■ 조건부분포가 정규분포

■ $X \sim N(\mu, \Sigma)$ 에서 $X = \begin{bmatrix} X_1 \\ X_2 \end{bmatrix}$, $\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}$, $\Sigma = \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix}$ 일 때,

$$X_1 | X_2 = x_2 \sim N(\mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2), \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21})$$

■ 주변분포가 정규분포

■ 확률 벡터의 일부 성분만 떼어 내도 여전히 정규분포

예) 위의 다변량 정규분포에서 $X_1 \sim N(\mu_1, \Sigma_{11})$, $X_2 \sim N(\mu_2, \Sigma_{22})$

■ 공분산과 독립의 관계

■ 공분산이 0이면 독립 보장

■ 아핀 변환의 닫힘

■ $Y = AX + b \sim N(A\mu + b, A\Sigma A^T)$

3

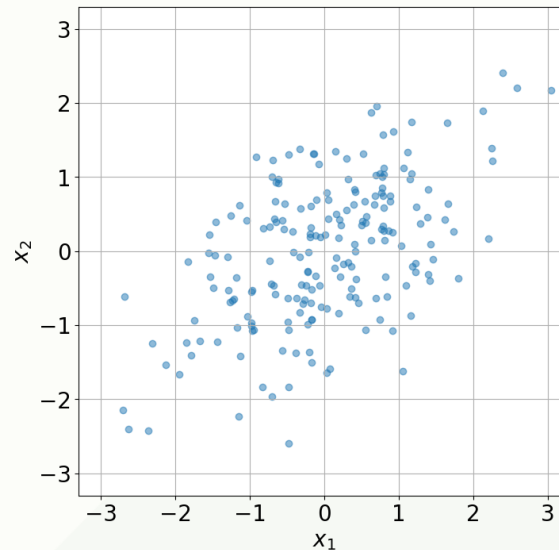
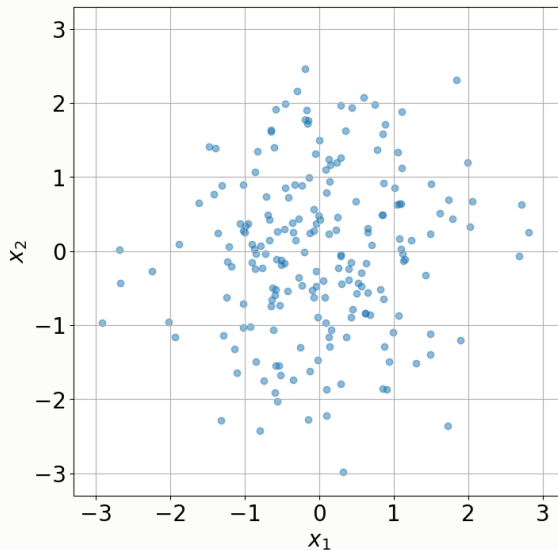
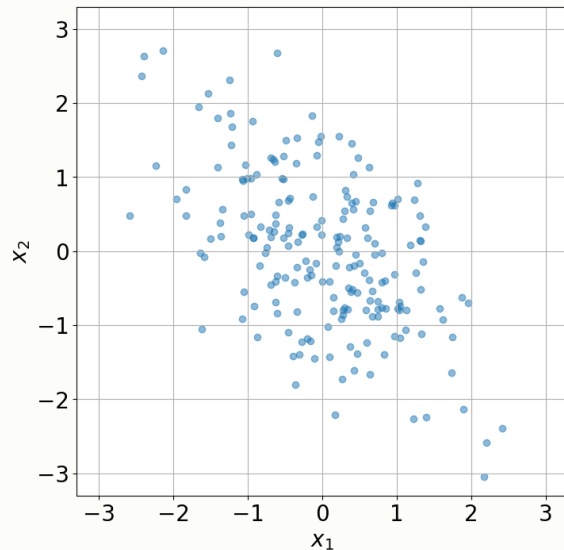
다변량 정규분포

● 다변량 정규분포와 공분산행렬

■ 이변량정규분포예제

■ 예) 200개 표본의 산점도

$$N\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & -0.6 \\ -0.6 & 1 \end{bmatrix}\right) / N\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\right) / N\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0.6 \\ 0.6 & 1 \end{bmatrix}\right)$$



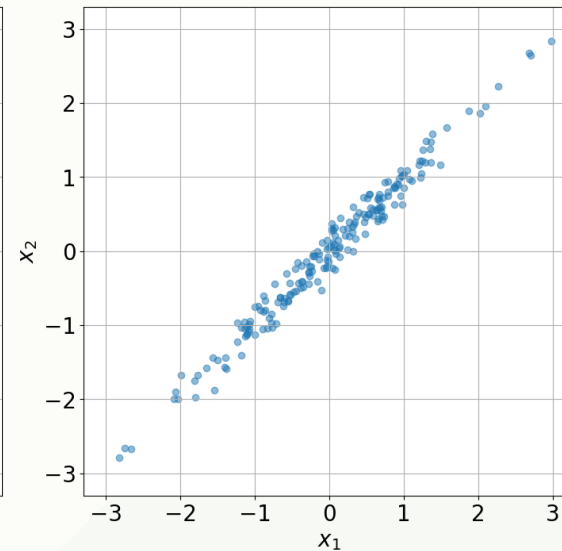
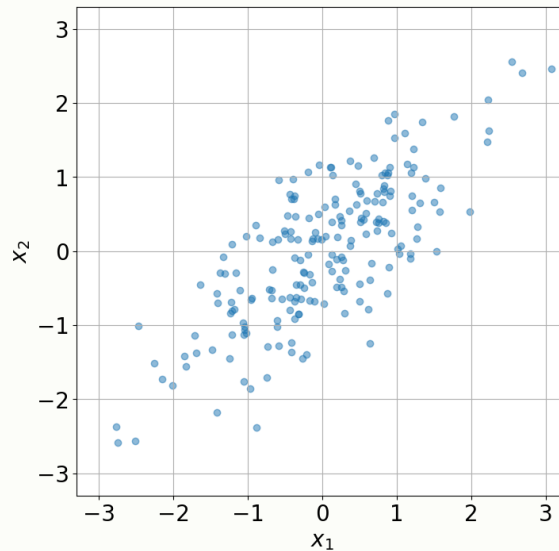
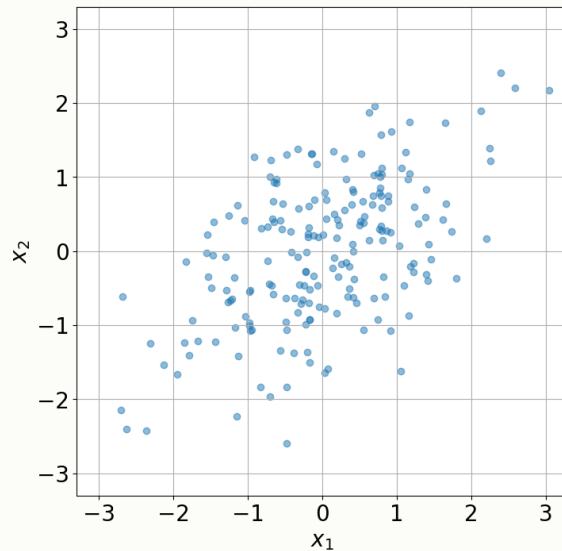
3 다변량 정규분포

● 다변량 정규분포와 공분산행렬

■ 이변량정규분포예제

■ 예) 200개 표본의 산점도

$$N\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0.6 \\ 0.6 & 1 \end{bmatrix}\right) / N\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}\right) / N\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0.99 \\ 0.99 & 1 \end{bmatrix}\right)$$



3 다변량 정규분포

● 양의 정부호 행렬(positive-definite matrix)

- 행렬 A 가 양의 정부호일 경우, 임의의 실수 벡터 $v \in \mathbb{R}^n$ 에 대하여

$$v^T A v > 0$$

- 양의 준정부호(positive-semidefinite)는 $v^T A v \geq 0$
- 양의 정부호와 랭크
 - 행렬 A 가 양의 정부호인 경우
 - 풀랭크행렬이며 역행렬 존재
 - 모든 고윳값 $\lambda_i > 0$ (대칭행렬)
 - 행렬 A 가 양의 준정부호이면서 정부호가 아닌 경우
 - 특이행렬이며 역행렬 존재안함
 - 모든 고윳값 $\lambda_i \geq 0$ (대칭행렬, 어떤 고윳값은 0)

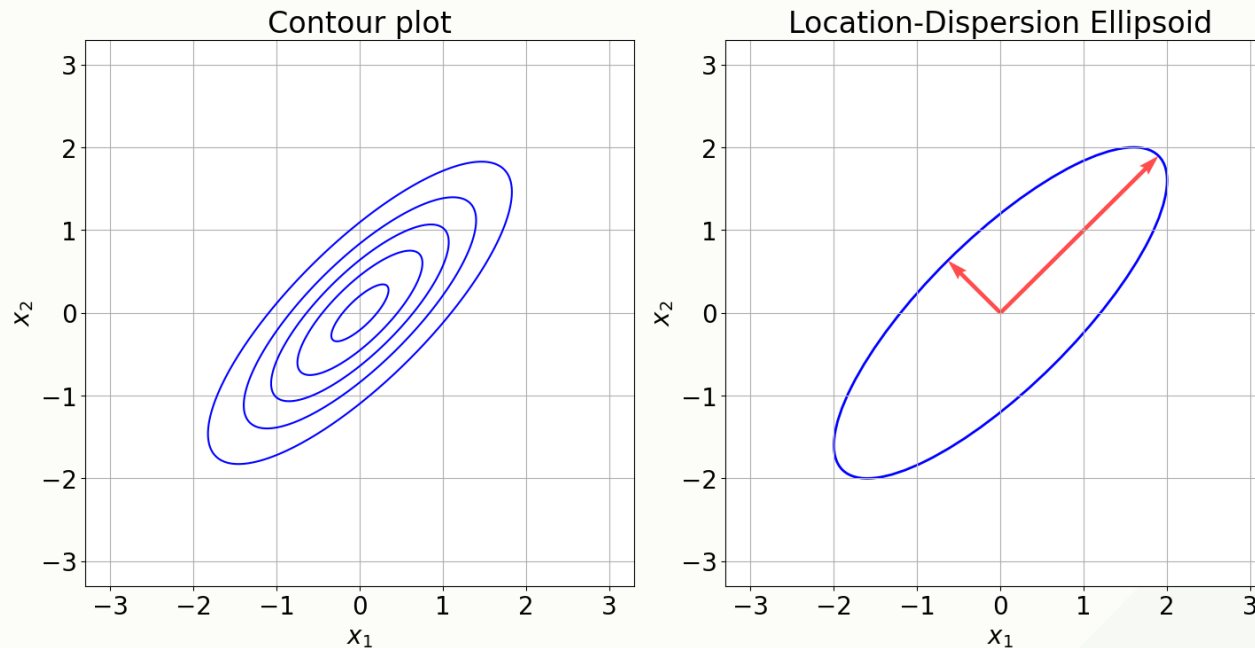
3 다변량 정규분포

● 공분산행렬과 고유값 분해

- 이변량정규분포의 공분산행렬에 대한 고유값 분해

$$\mathbf{X} = \begin{bmatrix} X_1 \\ X_2 \end{bmatrix} \sim N\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}\right)$$

- 결합확률밀도함수에 대한 등고선 그림



3 다변량 정규분포

● 공분산행렬과 고유값 분해

- 이변량정규분포의 공분산행렬에 대한 고윳값 분해

$$\mathbf{X} = \begin{bmatrix} X_1 \\ X_2 \end{bmatrix} \sim N\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}\right)$$

- 파이썬을 사용한 고윳값 분해 결과

```
eigenval, eigenvect = np.linalg.eigh(cov)
# 재건
reconst_cov = eigenvect @ np.diag(eigenval) @ eigenvect.T
print("Original Covariance Matrix:")
print(cov)
print("Eigenvalues (λ):")
print(eigenval)
print("Eigenvectors (U):")
print(eigenvect)
print("Reconstructed Covariance Matrix (UΛU^T):")
print(reconst_cov)
```

```
Original Covariance Matrix:
[[1.  0.8]
 [0.8 1. ]]
Eigenvalues (λ):
[0.2 1.8]
Eigenvectors (U):
[[-0.70710678  0.70710678]
 [ 0.70710678  0.70710678]]
Reconstructed Covariance Matrix (UΛU^T):
[[1.  0.8]
 [0.8 1. ]]
```

04

주성분 분석

4

주성분 분석

● 주성분 분석의 진행 과정

- 데이터 행렬에서 각 변수의 평균을 빼서 중심화(centering)
 - 표준화(standardization): 평균을 빼고 표준편차로 나눔
- 중심화된 데이터 행렬의 공분산 행렬 계산
- 공분산 행렬을 고윳값 분해
- 고윳값의 크기 순으로 상위 k 개의 주성분 선택
- 선택된 고유벡터로 원래 데이터를 투영하여 차원 축소

4 주성분 분석

● 주성분 분석 예제) iris 데이터

▪ iris 데이터 불러오기

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
# 데이터 불러오기
iris = load_iris()
X = iris.data          # (150, 4) - 150송이, 4개 변수
y = iris.target        # 0, 1, 2 - 3종류 붓꽃
print("데이터 크기:", X.shape)
print("변수 이름:", iris.feature_names)
print("앞 5개:\n", X[:5])
```

4 주성분 분석

● 주성분 분석 예제) iris 데이터

▪ iris 데이터 불러오기

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
```

데이터 크기: (150, 4)

변수 이름: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

앞 5개:

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
```

4 주성분 분석

● 주성분 분석 예제) iris 데이터

▪ 공분산행렬과 고윳값 분해

```
# 중심화 (평균 빼기)
X_centered = X - X.mean(axis=0)
# 공분산행렬
cov_matrix = np.cov(X_centered, rowvar=False)
print("공분산행렬:\n", np.round(cov_matrix, 2))
# 고윳값 분해
eigenval, eigenvect = np.linalg.eigh(cov_matrix)
# 큰 순서대로 정렬
idx = eigenval.argsort()[::-1]
eigenval = eigenval[idx]
eigenvect = eigenvect[:, idx]
print("고윳값:", np.round(eigenval, 2))
print("설명 비율:", np.round(eigenval / eigenval.sum() * 100, 1), "%")
```

공분산행렬:

```
[[ 0.69 -0.04  1.27  0.52]
 [-0.04  0.19 -0.33 -0.12]
 [ 1.27 -0.33  3.12  1.3 ]
 [ 0.52 -0.12  1.3   0.58]]
```

고윳값: [4.23 0.24 0.08 0.02]

설명 비율: [92.5 5.3 1.7 0.5] %

4 주성분 분석

● 주성분 분석 예제) iris 데이터

■ 주성분 분석 수행

```
# PCA 실행 - 2차원으로 축소
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_centered)
print("원래 차원:", X.shape[1])    # 4
print("축소 후:", X_pca.shape[1])  # 2
print("설명 비율:", np.round(pca.explained_variance_ratio_ * 100, 1), "%")
```

원래 차원: 4

축소 후: 2

설명 비율: [92.5 5.3] %

4 주성분 분석

● 주성분 분석 예제) iris 데이터

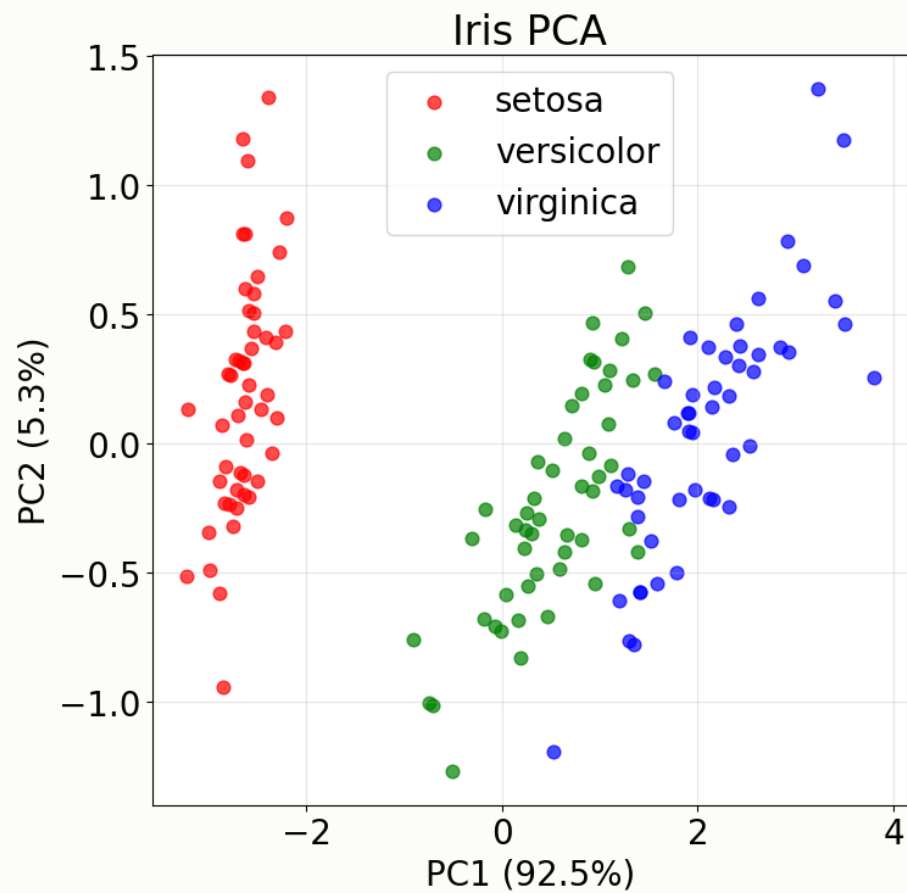
▪ 주성분 분석 결과 시각화

```
target_names = iris.target_names
colors = ['red', 'green', 'blue']
plt.figure(figsize=(8, 8))
for i, name in enumerate(target_names):
    mask = (y == i)
    plt.scatter(X_pca[mask, 0], X_pca[mask, 1],
                c=colors[i], label=name, alpha=0.7, s=60)
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)')
plt.title('Iris PCA')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```


4 주성분 분석

● 주성분 분석 예제) iris 데이터

■ 주성분 분석 결과 시각화



4 주성분 분석

● 주성분 분석 예제) iris 데이터

▪ 두 주성분(PC1, PC2)에 대한 해석

```
# 각 주성분에 대한 변수의 기여도 (loadings)
loadings = pca.components_
feature_names = iris.feature_names
print("PC1 구성:")
for name, val in zip(feature_names, loadings[0]):
    print(f" {name}: {val:.3f}")
print("\nPC2 구성:")
for name, val in zip(feature_names, loadings[1]):
    print(f" {name}: {val:.3f}")
```

- 주성분: 원래 변수들의 가중 합성 변수
 - PC1: 꽃잎(petal) 크기
 - PC2: 꽃받침(sepal) 크기

PC1 구성:

```
sepal length (cm): 0.361
sepal width (cm): -0.085
petal length (cm): 0.857
petal width (cm): 0.358
```

PC2 구성:

```
sepal length (cm): 0.657
sepal width (cm): 0.730
petal length (cm): -0.173
petal width (cm): -0.075
```

4 주성분 분석

● 주성분 분석 예제) iris 데이터

■ 주성분 수를 바꿔보기

```
pca3 = PCA(n_components=3)
X_pca3 = pca3.fit_transform(X_centered)
print("3개 주성분 누적 설명 비율:",
      np.round(pca3.explained_variance_ratio_.sum() * 100, 1), "%")
```

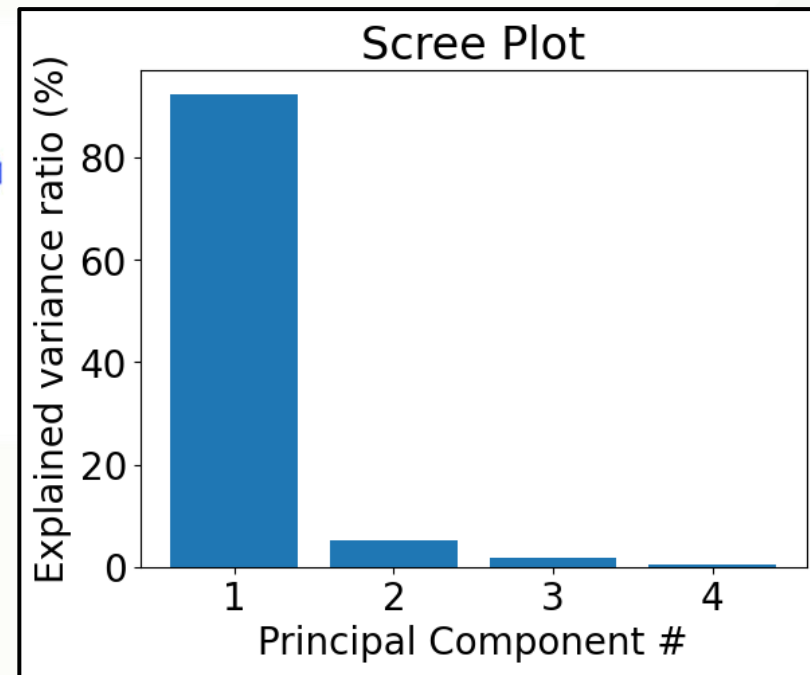
3개 주성분 누적 설명 비율: 99.5 %

4 주성분 분석

● 주성분 분석 예제) iris 데이터

- 주성분 수를 바꿔보기 Scree plot

```
pca_full = PCA()  
pca_full.fit(X_centered)  
plt.bar(range(1, 5), pca_full.explained_variance_ratio_ * 100)  
plt.xlabel('Principal Component #')  
plt.ylabel('Explained variance ratio (%)')  
plt.title('Scree Plot')  
plt.show()
```



감사합니다
